LLM-Assisted Mobile Robot Task Planning in Simulation: From Natural Language to Validated Skill Sequences

Ruijie Zhu
Department of Electromechanical Engineering
University of Macau, Macau, China
Email: mc65809@um.edu.mo

***Abstract*—We present a simulated mobile robot system that converts natural-language instructions into validated sequences of robot skills. A task planner, either rule-based or driven by a large language model (LLM) with OpenAI-compatible function calling, proposes calls to a fixed allowlist of navigation skills. A validator and executor enforce argument types, workspace bounds, mutual exclusion, and timeouts before the robot moves, and each skill returns a structured result with measurable error metrics that drive bounded recovery (retry once, reject, or stop and report). Experiments in Webots with a TurtleBot3 show all required requests succeed with position and heading errors within 0.1 m and 0.1 rad, invalid locations are rejected without moving the robot, and malformed LLM arguments are intercepted by the validator.**

I. Introduction

This work tackles the problem of turning natural-language instructions into safe, executable robot tasks. The system takes a request such as "*Go to storage, then visit the workbench, and stop.*", and produces a small plan, validated before execution, that the robot carries out safely in a Webots simulation.

Existing LLM-based robot systems have shown that language can guide action planning, but directly mapping unconstrained model outputs to robot execution is risky: an LLM may emit syntactically valid yet physically infeasible commands, invalid parameters, or sequences that do not terminate as expected. This creates a gap between the flexibility of high-level language reasoning and the reliability required by low-level robot execution. This work therefore separates high-level task planning from deterministic robot execution through a constrained skill interface and a validation layer: the model suggests, but deterministic rules decide what is executed.

The design is guided by four goals:

- **Safety:** the LLM may only *suggest* skill calls through a fixed allowlist of three skills. Every call is validated before it reaches the robot, and the LLM can never issue velocity commands or execute arbitrary code.
- **Explicit contracts:** each skill defines typed parameters with valid ranges, preconditions, a success condition with a timeout, and a structured result containing at least `success`, `reason`, and elapsed time.
- **Reliable navigation:** target reaching is reported only when the robot is within a stated position *and* heading tolerance. Timeouts are handled safely and reported with measurable error metrics.
- **Modularity:** the control layer (Part 2), the skill interface (Part 3), and the task planner (Part 4) are decoupled and testable without the simulator.

The project is organized into four parts: Part 1 sets up and verifies the simulation environment with manual keyboard control, Part 2 implements the navigation skills, Part 3 defines the structured skill interface, and Part 4 adds LLM-based task planning. The remainder of this paper follows this structure.

Related work

LLMs have recently been used for robot control: SayCan [1] and RT-2 [2] ground language in robotic policies, and Voyager [3] drives long-horizon tasks through tool calling. Code as Policies [4] goes further and lets the model generate executable Python control code directly. Those works focus on generalization, exploration, or code generation; the present work instead emphasizes safety: the LLM only proposes calls to a fixed allowlist of skills, and a validator with bounded recovery guards every execution before the robot moves. Unlike CaP, our executor never evaluates or runs code produced by the model.

II. System Architecture and Skill Interface

A. Architecture

The pipeline is shown in Fig. 1. A natural-language request is turned into a sequence of tool calls by a task planner that runs either a rule-based mock planner or a real LLM with OpenAI-compatible function calling. Every proposed call passes through a validator and the `SkillExecutor`, which enforces the allowlist, argument checks, workspace bounds, mutual exclusion, timeouts, and structured logging. Executed skills command the robot through `/cmd_vel` and read pose from `/odom`. The structured result of each skill is fed back to the planner for bounded recovery decisions. The key design principle is that the LLM has planning authority

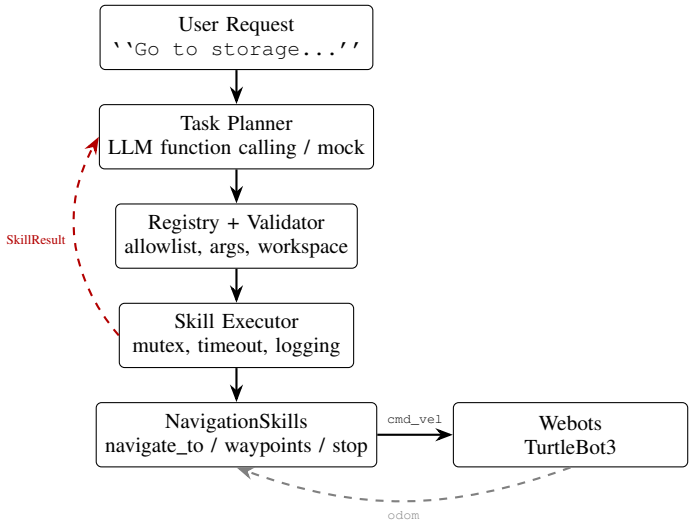

Fig. 1. System architecture: request → planner → validator/executor → skills → simulation, with structured results fed back for recovery.

but no direct execution authority: all robot-affecting actions are gated by deterministic validation and execution rules, so model variability cannot propagate to unsafe motion. This separation is intentional: the LLM is used where flexibility matters (interpreting instructions, composing skills), while robot-affecting operations stay in deterministic, testable components. The design trades some action-space flexibility for stronger controllability, validation, and reproducibility.

B. Skill interface

Each of the three skills (`navigate_to(x, y, theta)`, `follow_waypoints(waypoints)`, and `stop()`) is described by a registry entry containing a description, typed parameters with minimum/maximum bounds, preconditions, a success condition, and a timeout. The workspace bounds are set from the physical walls of the Webots TurtleBot3 house world (odom frame: west wall at $x = -1.36$, east wall at $x = 3.74$, south wall at $y = -5.1$, north wall at $y = 5.1$). The validator uses slightly extended limits ($x \in [-2, 4]$, $y \in [-5.2, 5.2]$) to provide a safety margin, and rejects any target outside this measured area before it reaches the robot. A call returns a `SkillResult` with a `reason` enumeration (`ok`, `timeout`, `invalid_arguments`, `precondition_failed`, etc.) plus measurable fields such as `position_error_m` and `heading_error_rad`. The same registry generates three machine-readable views (a compact registry for system prompts, OpenAI tool schemas for function calling, and standard JSON Schema), keeping the three views consistent with each other and with the validator.

III. Implementation

A. Navigation (Part 2)

`navigate_to` reports success only when the position error and the heading error are both within their tolerances (0.1 m and 0.1 rad). In the initial implementation the heading error was evaluated relative to the instantaneous direction from the robot to the target. This caused false successes: the robot could arrive at the target position facing the direction of travel instead of the user-specified heading `theta`. To fix this, the success condition now evaluates the heading error strictly against `target_theta`, and a separate in-place rotation phase is activated after the robot reaches the target position to align it with `target_theta` before success is reported.

The controller is therefore a two-phase state machine with separate control laws per axis: PD on the linear axis and a full PID on the angular axis. While the robot is farther than the position tolerance from the target, it first turns toward the target direction and then drives, with the speed scaled down near the goal. The linear command is $v = \mathrm{clamp}(0.35\,d + 0.08\,\dot{d},\ 0.05,\ 0.25)$ m/s, a proportional law on the remaining distance d plus a derivative term on its rate of change $\dot{d}$ that brakes early near the goal. The angular axis uses a discrete PID ($K_p = 1.0$, $K_i = 0.02$, $K_d = 0.03$) with an integral clamp of 0.5, a derivative limit of 1.5 rad/s, a dead-band of 0.02 rad, and a 0.15 rad/s minimum-output floor so that small commands remain effective despite wheel friction. While driving, small heading errors are corrected continuously with a bounded steering term instead of discrete stop-and-turn cycles, which removes the jerky motion observed when correction was only applied above a hard threshold. Once inside the position tolerance, the phase is locked into in-place rotation to the target heading, with hysteresis (0.25 m) so that small position drift does not make the robot oscillate between driving and turning. This failure mode was observed in an early version and caused timeouts. On timeout the robot stops and reports the remaining position and heading errors.

`follow_waypoints` visits each pose in order and reports `last_reached` and `failed_at` indices on failure, dividing its total timeout budget evenly among the waypoints. `stop()` publishes a zero twist several times to guarantee the robot halts.

B. Skill interface and executor (Part 3)

Part 3 acts as a structured skill interface that decouples the high-level task planner (Part 4) from the low-level control layer (Part 2). It provides three functions: contract definition through a skill registry that specifies names, typed parameters, preconditions, and timeouts; argument validation and workspace-bound checking that reject malformed or out-of-range calls before they reach the robot; and standardized result formatting that returns structured JSON (success, reason, elapsed time) for every execution, enabling the planner to make consistent recovery decisions. The interface layer is implemented with Python dataclasses (typed, JSON-serializable, zero external dependencies). The `SkillExecutor` is the only entry point to the robot: it checks the allowlist, validates arguments (types, ranges, workspace, and the structure of waypoints), enforces mutual exclusion (a second skill call while busy is rejected with `precondition_failed`, except `stop`),

applies an enforced timeout, and appends one JSON line per call (`logs/skill_calls.jsonl`).

C. LLM tool calling and safety (Part 4)

The planner calls an OpenAI-compatible API (DeepSeek `deepseek-chat`) with `tools=tool_schemas()`. The LLM only proposes calls. The loop executes each call through the executor and feeds the `SkillResult` back, repeating until the model replies with a final message. At most `max_steps` tool calls are allowed per task, and at most one retry per failing skill, so a misbehaving model cannot keep the robot busy indefinitely. The mock planner used during development emits the same call format, so the validator, executor, and recovery paths are exercised identically in offline tests without an API key. Recovery is rule-based and bounded: `timeout` → retry once → if it fails again, stop and report; `invalid_arguments` or unknown skills → reject the step and abort the plan. An early failure in which the model produced waypoints with missing headings was fixed by making each waypoint an object $\{x, y, theta\}$ in the tool schema, which the validator also enforces. Dispatch is restricted to the three registered methods. The executor never evaluates or executes code generated by the model. Every task run additionally records the original plan, per-step results, recovery decision, and final outcome in `logs/task_planner.jsonl`.

D. Validation and rejection classes

Calls are rejected with a typed reason before any command is published: `unknown_skill` for a call outside the fixed allowlist (for example `teleport`), `invalid_arguments` for wrong types, missing parameters, targets outside the measured workspace, or waypoints that are not {`x, y, theta`} objects, and `precondition_failed` when a second call arrives while the robot is busy. These classes map one-to-one to the required safety checks (allowlist, typed arguments, workspace bounds, and bounded step and retry counts). Each rejection leaves the robot stationary. One live case, waypoints generated without `theta`, is discussed in Section IV.

IV. Experiments and Results

A. Setup

Experiments ran in an Ubuntu 22.04 virtual machine with ROS 2 Humble and the Webots *TurtleBot3 Burger* house world. Basic motion was first verified by keyboard teleoperation in the same environment (Part 1). The robot spawns at the origin of the odometry frame. Four named locations (`entrance`, `storage`, `workbench`, `charging_station`) were placed in obstacle-free areas verified by a clearance checker over all pairwise straight-line paths (≥ 0.35 m from all walls and furniture). Because navigation follows straight lines without obstacle avoidance, this manual placement guarantees that every commanded path is collision-free by construction. The LLM used is DeepSeek `deepseek-chat` [5] through the OpenAI-compatible API [**?**].

TABLE I
Navigation accuracy in the Webots simulator (Part 2).

Target (pose)	Elapsed (s)	pos err (m)	hdg err (rad)
entrance $(-0.7, -1.6, \pi)$	17.4	0.086	0.099
storage $(0.0, -3.2, 0)$	17.5	0.087	0.090
workbench $(1.5, -2.8, \pi/2)$	14.9	0.086	0.086
charging_station $(2.2, -3.5, 0)$	13.8	0.084	0.096

Note: each named location was reached within the `navigate_to` tolerances (0.1 m, 0.1 rad) in the Webots simulator. Measured errors across these runs were 0.084–0.087 m (position) and 0.086–0.099 rad (heading). Additional task-level runs (see Table II) measured position errors up to 0.099 m and heading errors within the same 0.086–0.099 rad band.

TABLE II
Results in the Webots simulator.

Request (mode)	Outcome	Time (s)	Recovery
Go to storage (mock)	success	24.5	—
Workbench, storage, stop (mock)	success	31.3	—
Go to kitchen (mock)	rejected	0.0	reject
Go to storage (LLM)	success	26.5	retry once
Workbench, storage, stop (LLM)	success	34.4	—
Go to kitchen (LLM)	rejected	0.5	reject
timeout scenario (offline)	failed	0.8	retry, stop+report

Note: in the LLM run of `Go to storage.`, the first navigation attempt timed out (20.2 s) and the planner retried once, which succeeded; the retry is therefore listed as a recovery action although the task outcome is success.

B. Test cases

Table I reports the navigation accuracy of the Part 2 skills in the Webots simulator: reaching a target is reported only when the position error is ≤ 0.1 m *and* the heading error is ≤ 0.1 rad, including the heading-alignment case ($\theta = \pi/2$) that exercises the in-place rotation phase.

Table II evaluates the main links in the execution pipeline rather than only the end-to-end success rate. The single-location request exercises natural-language grounding and basic skill selection, the multi-location request exercises sequential task planning and execution, the unknown-location case exercises pre-execution validation, and the timeout scenario exercises bounded recovery. The malformed-waypoint case (Section IV) further tests whether schema-level errors are intercepted before reaching the robot. Together these cases cover planning, execution, validation, and recovery within the implemented system.

C. Analysis

All three required requests succeed in both modes. The robot reaches every target with position error ≤ 0.1 m and heading error ≤ 0.1 rad (e.g., `position_error_m` $= 0.087$, `heading_error_rad` $= 0.099$ for the workbench in the LLM run), which validates the heading-alignment fix. The invalid location "kitchen" is rejected without moving the robot: the mock planner lists valid locations, and the LLM independently refuses with "invalid location: kitchen". The bounded recovery path was exercised: after a navigation timeout the planner retried once, and when the retry also failed it stopped and reported. A second failure case, the LLM

emitting waypoints with missing headings, was intercepted by the validator (`invalid_arguments` → `reject_step`), and fixing the tool schema to object-form waypoints made the same request succeed. This case shows that the validator catches such failures before any command is sent. Repeated runs of the same instruction were consistent: the mock planner's `Go to storage.` completed in 24.5 and 24.7 s in two sessions and in 26.5 s with the LLM. The small difference is expected: the LLM path adds API round-trip latency, and for the multi-location request the LLM may choose a different (but equivalent) skill sequence such as a single `follow_waypoints` call instead of two `navigate_to` calls, so wall-clock time is not directly comparable between modes. Every run converged within the stated tolerances, with measured position errors in 0.084–0.099 m and heading errors in 0.086–0.099 rad. For every task the planner log records the original plan, per-step results, recovery decision, and final outcome, which supports auditing and reproduction. Taken together, these results suggest that the validator–executor boundary decouples LLM output variability from low-level robot safety, while retaining enough flexibility for the required natural-language tasks: the model can plan freely, but every action that reaches the robot is deterministic, validated, and logged.

The mock-versus-LLM comparison also illustrates the underlying engineering trade-off. The mock planner is deterministic and fast, but limited to predefined task patterns; the LLM planner adds latency and nondeterminism in exchange for natural-language flexibility and variable skill sequences. The role of the validator–executor boundary is therefore not to remove this variability but to contain it at the planning layer, keeping robot execution constrained, observable, and recoverable.

V. Conclusion and Future Work

We built and validated a safe, modular LLM-assisted robot task planning system in simulation. Every proposed skill call passes an executor gate (allowlist, argument validation, preconditions, guarded execution, structured results with logging) before the robot moves. In Webots experiments all required requests succeeded within 0.1 m position and 0.1 rad heading error, and invalid or out-of-range requests were rejected without robot motion. The main contribution is a reproducible engineering architecture that combines function-calling LLMs with explicit skill contracts and bounded recovery, yielding a system that is safe in practice and reproducible.

Recalling the four design goals from Section I, the system achieves safety by confining the LLM to a fixed allowlist and validating every call before execution; explicit contracts through the registry's typed parameters, preconditions, and structured results; reliable navigation by reporting success only within the position and heading tolerances and handling timeouts safely; and modularity by decoupling the control, interface, and planning layers so that each part can be tested independently. The evaluation provides evidence that these goals are supported in the implemented system: the required requests succeed within tolerance, and invalid or out-of-range requests never move the robot.

The current system has no obstacle avoidance: navigation follows straight lines, so locations must be manually selected in obstacle-free areas. A lidar-based local avoidance or a navigation stack would remove this constraint. Locations are hard-coded instead of discovered from a map, and the planner is single-request and stateless. LLM behavior is nondeterministic, so the validator must remain strict. Recovery is rule-based and does not yet let the model repair its own plan. A further direction is state-aware validation: the current validator checks syntax, ranges, the allowlist, and the workspace, but a legally typed argument is not necessarily feasible in the robot's current state; checking the robot's pose and path feasibility against an obstacle map before execution would tighten the safety boundary once such a map becomes available. Future work also includes occupancy-map based planning, multi-request stateful dialogues, learned recovery policies, and migration to a physical robot, since the executor already accepts any robot driver that provides the three skill methods. As a safety consideration, API credentials are supplied through environment variables only and are never stored in the repository.

References

[1] M. Ahn et al., "Do as I Can, Not as I Say: Grounding Language in Robotic Affordances," arXiv:2204.01691, 2022. [Online]. Available: https://arxiv.org/abs/2204.01691

[2] A. Brohan et al., "RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control," arXiv:2307.15818, 2023. [Online]. Available: https://arxiv.org/abs/2307.15818

[3] G. Wang et al., "Voyager: An Open-Ended Embodied Agent with Large Language Models," arXiv:2305.16291, 2023. [Online]. Available: https://arxiv.org/abs/2305.16291

[4] J. Liang et al., "Code as Policies: Language Model Programs for Embodied Control," arXiv:2209.07753, 2022. [Online]. Available: https://arxiv.org/abs/2209.07753

[5] DeepSeek, "API Documentation." [Online]. Available: https://api-docs.deepseek.com/ (Accessed: Sep. 2026)